



Core Data

Wrapper over SQLite

What is Core Data?

→ graph made up of nodes (entities) and edges (connections)
→ sqlite, xml, binary files

- Core Data is Apple's object graph and persistence framework used to manage the model layer of an application
- It provides a way to store, retrieve, update, delete, and manage application data efficiently
- Important myths about core data
 - Core Data is not a database
 - Core Data is not SQLite
 - SQLite is simply one of the storage options that Core Data can use
- The key concept to understand is that Core Data is not merely a database like SQLite or MySQL
- While it can use SQLite as a persistent store behind the scenes, you don't interact with tables, rows, or SQL directly
- Instead, it is fundamentally an object graph management system
- Core Data works as an Object-Relational Mapping (ORM) framework similar to Hibernate (Java), Entity Framework (.NET), or SQLAlchemy (Python)

Why Core Data



- Without Core Data, developers have to:
 - Write SQL queries
 - Manage relationships manually
 - Handle object serialization
 - Track changes
 - Implement caching
 - Handle undo/redo functionality
- Core Data performs these tasks automatically

Features



- Persistent storage : uses SQLite DB
- Object graph management : similar to ORM
- Automatic change tracking : possible to rollback
- Lazy loading (Faulting) : memory efficiency
- Data validation
- Relationship management
- Query optimization : No way to write direct SQL queries
- Undo/Redo support : rollback and savepoint
- Batch operations : performance while execution
- Data migration
- Versioning

Benefits & Drawbacks



■ Benefits

- First-party Apple framework, well-integrated with iOS ecosystem
- Automatic change tracking and undo/redo support
- Lazy loading and caching for efficient memory usage
- Visual data model editor reduces boilerplate → .xcmodeldata file
- Built-in validation and relationships

Entity
attributes
relationships

■ Drawbacks

- Not designed for remote/online data storage (by itself)
- Steep learning curve—many moving parts
- Not a traditional SQL database; thinking in terms of object graphs is different



Key Components

NSManagedObjectModel : Schema of db → definitions of entities | attributes | relationships



- An in-memory representation of your data model - the complete schema definition

- What it contains

- ✓ Entities - Descriptions of data types (e.g., Person, Book, Author) → tables inside SQLite
- ✓ Attributes - Properties of entities (String, Int, Date, Boolean, etc.) → columns of a table
 - Relationships - Connections between entities (1-to-1, 1-to-many, many-to-many)
- ✓ Fetch Requests - Predefined queries saved in the model → request to read the data
- Configurations - Groups of entities that can be stored in different stores

- How it's created

- Visual editor in Xcode (.xcdatamodeld file) → preferred
- Programmatically using NSManagedObjectModel API

- The model defines what data can be stored and how objects relate to each other

Changes to the model require migration strategies

```
// Access the model
let model = persistentContainer.managedObjectModel

// Get all entities
let entities = model.entities

// Get a specific entity
let personEntity = model.entitiesByName["Person"]

// Create model programmatically
let model = NSManagedObjectModel()
let entity = NSEntityDescription()
entity.name = "Person"
model.entities = [entity]
```

NSManagedObjectContext

- The most critical component you'll interact with. It's a temporary workspace where objects live and changes are tracked before being saved to disk

- Primary responsibilities

- Object lifecycle management - Create, fetch, update, delete objects
- Change tracking - Knows what objects are dirty (changed, inserted, deleted)
- Undo/Redo support - Built-in undo manager → rollback
- Validation - Ensures data meets model constraints
- Merge policies - Handles conflicts when multiple contexts save

→ NSManagedObject
Read | RW

- Core Properties

```
// Main thread context
```

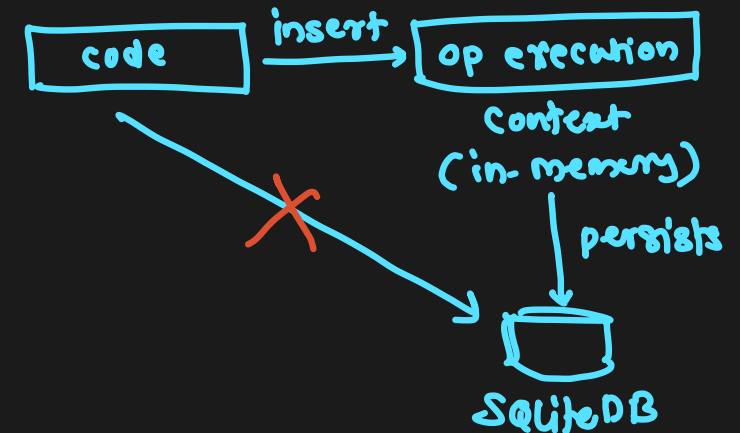
```
let context = persistentContainer.viewContext
```

```
// Configuration
```

```
context.mergePolicy = NSMergeByPropertyObjectTrumpMergePolicy
```

```
context.undoManager = UndoManager()
```

```
context.automaticallyMergesChangesFromParent = true // iOS 10+
```



NSManagedObjectContext



▪ Core Operations

call save() after every → insert / update / delete
✓ try context.save() → *persist data in SQLite DB*

// Rollback changes

✓ context.rollback()

// Reset context (clears all objects)

✓ context.reset()

// Get object by ID

let object = try context.existingObject(with: objectID)

// Refresh object (revert to saved state)

context.refresh(object, mergeChanges: false)

▪ Thread Safety Rules

- The viewContext belongs to the main thread only
- For background work, create a private context

NSPersistentStoreCoordinator

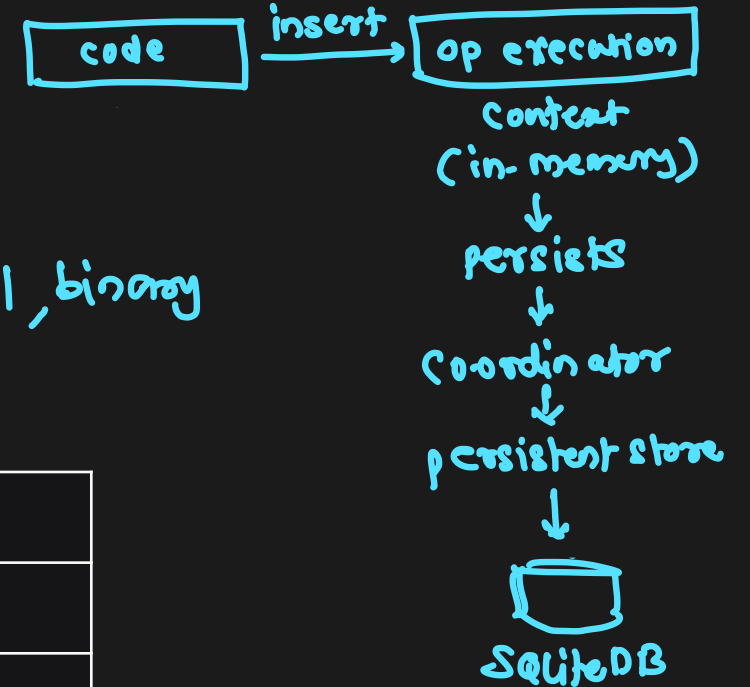
- The bridge between the managed object context and the persistent store(s). It ensures that objects are saved to the correct store and loaded from the right place

- **Key responsibilities**

- **Store management** - Adds/removes persistent stores
- **Object distribution** - Routes objects to appropriate stores based on configurations
- **Schema migration** - Handles lightweight migrations
- **Store coordination** - Manages access to the underlying data files → SQLite, XML, binary

- **Store types**

Store Type	Description	Use Case
<u>NSSQLiteStoreType</u>	<u>SQLite database</u>	<u>Production (most common)</u>
<u>NSBinaryStoreType</u>	<u>Binary file</u>	<u>Small datasets, fast read/write</u>
<u>NSInMemoryStoreType</u>	<u>RAM only</u>	Testing, temporary data
<u>NSXMLStoreType</u>	<u>XML file</u>	<u>macOS only, human-readable</u>



default

NSPersistentContainer



- Introduced in iOS 10/macOS 10.12, this is a unified wrapper that creates and manages the entire Core Data stack automatically
- Why use it
 - Reduces boilerplate code
 - Automatically creates the stack
 - Manages context hierarchy (parent-child contexts)
 - Simplifies background operations

```
class AppDelegate: UIResponder, UIApplicationDelegate {

    lazy var persistentContainer: NSPersistentContainer = {
        let container = NSPersistentContainer(name: "DataModel")
        container.loadPersistentStores { (description, error) in
            if let error = error {
                fatalError("Failed to load Core Data stack: \(error)")
            }
        }
        return container
    }()

    func saveContext() {
        let context = persistentContainer.viewContext
        if context.hasChanges {
            do {
                try context.save()
            } catch {
                let nserror = error as NSError
                fatalError("Unresolved error \(nserror)")
            }
        }
    }
}
```

NSManagedObject



- The base class for all Core Data objects. Think of it as a record or row in database terms.
- **Key features**
 - All properties are stored as @NSManaged (dynamic dispatch)
 - Properties can be optional or non-optional
 - Automatically generated getters/setters interact with the context
 - Supports custom subclassing with validation logic

```
@objc(Person)
public class Person: NSManagedObject {
    @NSManaged public var name: String?
    @NSManaged public var age: Int16
    @NSManaged public var birthDate: Date?
    @NSManaged public var books: NSSet? // To-many relationship
}

extension Person {
    @nonobjc public class func fetchRequest() -> NSFetchedResultsController {
        return NSFetchedResultsController(entityName: "Person")
    }

    // To-many relationship helpers
    @objc(addBooksObject:)
    @NSManaged public func addToBooks(_ value: Book)

    @objc(removeBooksObject:)
    @NSManaged public func removeFromBooks(_ value: Book)
}
```

NSFetchRequest (The Query)



- Describes what data to fetch from the persistent store

```
let request = NSFetchRequest<Person>(entityName: "Person")

// Sorting
request.sortDescriptors = [
        NSSortDescriptor(key: "name", ascending: true)
]

// Filtering (Predicate)
request.predicate = NSPredicate(format: "age > %d AND name CONTAINS[c] %@", 18, "alex")

// Limit results
request.fetchLimit = 50
request.fetchOffset = 10 // For pagination

// Batch size (reduces memory usage)
request.fetchBatchSize = 20
```



NSPredicate

- Defines conditions for fetching or filtering objects → where clause in SQL Query
- **Comparison operators**

Operator	Meaning	Example
<u>==</u>	<u>Equal</u>	<u>name == "Alice"</u>
<u>!=</u>	<u>Not equal</u>	<u>age != 30</u>
<u>> / <</u>	<u>Greater/less than</u>	<u>age > 18</u>
<u>>= / <=</u>	<u>Greater/less or equal</u>	<u>age >= 18</u>
<u>IN</u>	<u>In set</u>	<u>name IN {"Alice", "Bob"}</u>
<u>BETWEEN</u>	<u>Range</u>	<u>age BETWEEN {18, 65}</u>

NSPredicate



■ String comparisons

- BEGINSWITH - Starts with
- ENDSWITH - Ends with
- CONTAINS - Contains substring
- [c] - Case insensitive (BEGINSWITH[c])
- [d] - Diacritic insensitive
- [cd] - Case and diacritic insensitive

■ Aggregate operators

- @count - Count of relationship
- @sum - Sum of attribute
- @avg - Average
- @min / @max - Minimum/Maximum

```
// Complex predicate
```

```
let predicate = NSCompoundPredicate(type: .or, subpredicates: [  
    NSPredicate(format: "age < 18"),  
    NSPredicate(format: "age > 65")  
)
```

```
// Predicate with variable
```

```
let age = 21  
let predicate = NSPredicate(format: "age == %d", age)
```

```
// Predicate with dictionary
```

```
let dictionary = ["name": "Alice", "age": 25]  
let predicate = NSPredicate(format: "name == %@ AND age == %@",  
    dictionary["name"]!, dictionary["age"]!)
```

Common predicates



// Comparison

```
NSPredicate(format: "age >= %d", 21)
```

```
NSPredicate(format: "name == %@", "Alice")
```

```
NSPredicate(format: "name IN %@", ["Alice", "Bob"])
```

// String operations

```
NSPredicate(format: "name BEGINSWITH[c] %@", "a")
```

```
NSPredicate(format: "name CONTAINS[c] %@", "alex")
```

// Relationships

```
NSPredicate(format: "books.@count > 0")
```

```
NSPredicate(format: "ANY books.title == %@", "Swift Programming")
```

// Compound

```
NSPredicate(format: "age > 18 AND name != %@", "Admin")
```

// Subqueries

```
NSPredicate(format: "SUBQUERY(books, $book, $book.pages > 100).@count > 0")
```


Mapping / terminology

SQL

Core Data

- ① Database → NSManagedObjectContext (schema)
- ② Table → NSEntity (entity)
- ③ Column of table → Attribute of an entity
- ④ Record / Row → NSManagedObject
- ⑤ select query → NSFetchRequest
 - ↓
 - Where clause → NSPredicate
 - ↓
 - sort by → SortDescription



CRUD Operations

Setup Core Data in App Delegate → automatically



```
import CoreData
```

```
class AppDelegate: UIResponder, UIApplicationDelegate {
```

```
    lazy var persistentContainer: NSPersistentContainer = {  
        let container = NSPersistentContainer(name: "YourProjectName")  
        container.loadPersistentStores { (description, error) in  
            if let error = error {  
                fatalError("Failed to load Core Data: \(error)")  
            }  
        }  
        return container  
    }()  
}
```

↪ persists the data in persistent store (sqlite)

```
func saveContext() {  
    let context = persistentContainer.viewContext  
    if context.hasChanges {  
        do {  
            try context.save()  
        } catch {  
            let nserror = error as NSError  
            fatalError("Unresolved error \(nserror)")  
        }  
    }  
}
```

User (Managed Object)



```
import Foundation
```

```
import CoreData
```

→ Entity Name

```
@objc(User)
```

```
public class User: NSObject {
```

```
@NSManaged public var name: String?
```

```
@NSManaged public var email: String?
```

```
@NSManaged public var age: Int16
```

```
@NSManaged public var createdAt: Date?
```

} attributes

```
@nonobjc public class func fetchRequest() -> NSFetchRequest<User> {  
    return NSFetchRequest<User>(entityName: "User")  
}
```

```
}
```

CREATE - Add New User



```
import CoreData
import UIKit
```

```
// Get context
```

```
let appDelegate = UIApplication.shared.delegate as! AppDelegate
```

```
let context = appDelegate.persistentContainer.viewContext → get the context
```

```
// Create new user
```

```
let newUser = User(context: context)
```

```
newUser.name = "John Doe"
```

```
newUser.email = "john@example.com"
```

```
newUser.age = 25
```

```
newUser.createdAt = Date()
```

} create an object of managed object
and set the attribute values

```
// Save
```

```
do {
```

```
    try context.save()
```

```
    print("✅ User created successfully")
```

```
} catch {
```

```
    print("❌ Failed to create user: \(error)")
```

```
}
```

↪ persist the data

READ - Fetch All Users



```
import CoreData
```

```
import UIKit
```

```
// Get context
```

```
let appDelegate = UIApplication.shared.delegate as! AppDelegate
```

```
let context = appDelegate.persistentContainer.viewContext
```

```
// Create fetch request
```

↪ NSManaged object

```
let request: NSFetchedRequest<User> = User.fetchRequest()
```

```
// Optional: Add sorting
```

```
request.sortDescriptors = [NSSortDescriptor(key: "name", ascending: true)]
```

```
// Fetch
```

```
do {
```

↪ Execute select query

```
    let users = try context.fetch(request)
```

```
    print("📁 Total users: \(users.count)")
```

↪ managed object

```
    for user in users {
```

```
        print("👤 Name: \(user.name ?? "No Name"), Email: \(user.email ?? "No Email"), Age: \
```

```
        \
```

```
    } catch {
```

```
        print("❌ Failed to fetch: \(error)")
```

```
    }
```

UPDATE - Modify User



```
import CoreData
import UIKit

// Get context
let appDelegate = UIApplication.shared.delegate as! AppDelegate
let context = appDelegate.persistentContainer.viewContext

// Find user to update
let request: NSFetchRequest<User> = User.fetchRequest()
request.predicate = NSPredicate(format: "email == %@", "john@example.com")

do {
    let users = try context.fetch(request)

    if let user = users.first {
        // Update properties
        user.name = "Johnathan Smith"
        user.age = 26

        // Save
        try context.save()
        print("✅ User updated successfully")
    } else {
        print("❌ User not found")
    }
} catch {
    print("❌ Failed to update: \(error)")
}
```

DELETE - Remove User



```
import CoreData
import UIKit

// Get context
let appDelegate = UIApplication.shared.delegate as! AppDelegate
let context = appDelegate.persistentContainer.viewContext

// Find user to delete
let request: NSFetchRequest<User> = User.fetchRequest()
request.predicate = NSPredicate(format: "email == %@", "john@example.com")

do {
    let users = try context.fetch(request)

    if let user = users.first {
        // Delete
        context.delete(user)

        // Save
        try context.save()
        print("✅ User deleted successfully")
    } else {
        print("❌ User not found")
    }
} catch {
    print("❌ Failed to delete: \(error)")
}
```